

CaptionAI - Model Improvement Report

Project: AI Image Caption Generator (DenseNet201 encoder + LSTM/Attention decoders, Flickr8K dataset). Scope of this report: every algorithmic change made on top of the basic trained model, the reason for each change, and its measured effect on validation metrics (full held-out split: 1,214 images; 6,877 train / 8,091 total images; 40,455 captions). All numbers below are actual measured outputs stored in the repository (weights/metadata.json, experiments/*), not estimates.

Executive Summary

Starting from a correctly trained CNN+LSTM baseline (BLEU-4 = 0.1253), a two-tier optimisation program lifted caption quality to BLEU-4 = 0.1663 - a +33% relative improvement - while staying entirely within the Flickr8K dataset. The winning configuration is an attention decoder over spatial DenseNet201 features, trained with subword (BPE) targets, weight decay, embedding dropout, weight averaging (EMA) and scheduled sampling. It is now the production provider behind the web app.

Metric (val split, 1,214 imgs)	Baseline (start)	Final (production)	Change
BLEU-1	0.5334	0.5649	+5.9% rel
BLEU-2	0.3203	0.3704	+15.6% rel
BLEU-3	0.1890	0.2447	+29.4% rel
BLEU-4	0.1218	0.1663	+36.5% rel
ROUGE-L	0.2216	0.2548	+15.0% rel
Next-token top-1 accuracy	27.1%	30.9% (v2-global)	+3.8 pts
Confidence honesty (ECE)	measured: 1.3%	1.3% (already calibrated)	verified

1. Starting Point - The Baseline Model

1.1 Architecture and training setup

Component	Baseline configuration
Encoder	DenseNet201 (ImageNet weights, head removed, GlobalAvgPool), frozen -> 1920-d vector per image
Decoder	Dense(256) -> Embedding(8427, 256) -> concat -> LSTM(256) -> Dropout(0.5) -> residual Add -> Dense(128) -> Dropout(0.5) -> Dense(vocab, softmax)
Tokenizer	Word-level Keras Tokenizer, vocab 8,427, max caption length 35
Preprocessing	resize 224x224, scale 1/255 (no ImageNet normalisation)
Training	PyTorch on GTX 1080: Adam lr=1e-3, grad clip 5.0, label smoothing 0.1, batch 512, EarlyStopping patience 20
Decoding (served)	Beam search width 5, GNMT length penalty alpha=0.6, 2-gram repeat penalty x0.4

1.2 Baseline results and diagnosed weaknesses

Metric	Greedy	Beam-5 (served)
BLEU-1	0.4373	0.5334
BLEU-2	0.2600	0.3203
BLEU-3	0.1495	0.1890
BLEU-4	0.0959	0.1218
ROUGE-L	0.1933	0.2216
2-gram repetition	high (loop-prone)	2.80%

Diagnosed weaknesses: (1) Overfitting wall at epoch 26 - validation loss bottomed at 4.7517 (epoch 26 of 46) while training loss kept falling to 3.86; the model began memorising image-specific quirks, partly driven by thousands of hapax (once-only) words in the 8,427-word vocabulary that act as memorisation targets. (2) No spatial information - the 1920-d global average pooling discards object locations, hurting counts and relations. (3) Plain Adam, no weight decay, no embedding regularisation - nothing slowed memorisation. (4) Confidence display misreading - the UI showed ~28-35% 'confidence', which looked like poor accuracy but was in fact mean softmax probability over an 8,427-way choice.

2. Tier 1 - No-Retrain Optimisations

2.1 Beam search hyperparameter sweep

Swept 16 configurations (width {3, 5, 7, 10} x GNMT length penalty alpha {0.6, 0.8, 1.0, 1.2}) on the full validation split using a PyTorch replica of the served Keras model (numerically verified: the width-5 / alpha-0.6 row reproduced the published baseline bit-for-bit). Winner: width 10, alpha 1.2. Integrated into the FastAPI backend with environment overrides (BEAM_WIDTH, LENGTH_PENALTY).

Config	BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE-L	ms/image (GPU)
beam-3, alpha 0.6 (worst beam)	0.5290	0.3170	0.1859	0.1178	0.2210	9.0
beam-5, alpha 0.6 (old production)	0.5334	0.3203	0.1890	0.1218	0.2216	16.2
beam-10, alpha 1.2 (new production)	0.5358	0.3247	0.1947	0.1253	0.2247	35.9

Effect: BLEU-4 +2.8% relative (0.1218 to 0.1253) and every other metric up, with zero retraining. Also established that wider beams beyond 10 give diminishing/negative returns for this decoder.

2.2 Attention model (v1) forensic diagnosis

The earlier Phase-5 attention experiment had recorded good teacher-forced validation loss (3.68) but terrible generated captions (BLEU-1 0.38, BLEU-4 0.019). Two root causes were found: (a) Serving bug - the inference encoder applied the final DenseNet201 BatchNorm ('norm5') plus an extra ReLU although training consumed pre-norm5 features, i.e. the decoder received inputs from a distribution it had never seen. Fixed in inference/attention_provider.py. (b) Destroyed checkpoints - every surviving checkpoint was junk (attention_best.pt = epoch 1; attention_last.pt.bak = epoch 1; attention_last.pt = epoch 41 of a worse rerun). The genuinely trained weights had been overwritten by aborted resume runs. Verdict: unrecoverable without retraining - which motivated Tier 2 Step C with safe, strictly-improving checkpoint saving under experiments/.

2.3 Confidence calibration study

Teacher-forced all 62,515 validation (prefix -> next-token) pairs through the decoder and measured whether displayed softmax probability predicts correctness. Fitted temperature scaling by minimising token NLL over a T grid: optimal $T^* = 1.0$ exactly, expected calibration error ECE = 1.3%, mean displayed confidence 27.8% vs empirical next-token accuracy 27.1%. Conclusion: the number users see is already honest ('when the UI says X%, the word choice is right about X% of the time'); the apparent lowness is structural - label smoothing 0.1 caps certainty and the task has 8,427 choices (random = 0.012%). Action: documentation/UI tooltip instead of a code change; chasing a bigger percentage would reward bland captions, not better ones.

2.4 Independent semantic scoring (BLIP PMI)

Added a second opinion on every caption using the locally cached BLIP vision-language model: PMI = $\log P(\text{caption} | \text{image})$ minus $\log P(\text{caption} | \text{neutral grey image})$. The subtraction cancels the reward for generic phrasing - a raw probability scorer was shown in testing to rank a wrong generic sentence ('a blue car parked on the street') above the true specific one; PMI corrects this. Validated on Flickr8K samples (true image captions score PMI about +0.5..+1.9; wrong-image captions about +0.3..+0.7), then centred on the measured distribution of generated captions (median PMI +0.33, 150 images). Exposed as API fields semantic_score (logistic-mapped 0-100%) and semantic_pmi, rendered as a 'Semantic match' bar in the dashboard. Effect: catches fluent-but-wrong captions that softmax confidence alone misses (example: 'black dog is in the air in its mouth' scored 34.8% decoder confidence but only 6.9% semantic agreement).

3. Tier 2 - Retraining Within Flickr8K

3.1 Step A: BPE subword tokenizer (6,000)

Replaced the word-level vocabulary with a 6,000-entry ByteLevel-BPE tokenizer fitted on the training split only (34,385 captions; special ids PAD=0, START=1, END=2). Every one of the 40,455 captions round-trips encode->decode exactly (0 failures); frequent words stay single tokens (compression 0.965 tokens/word), rare words decompose into learnable pieces ('kayakers' -> 'kayak'+ers'), eliminating unlearnable hapax targets. Max encoded length 36, so sequences use MAX_LENGTH 38. Rationale: removes the memorisation shortcut behind the epoch-26 wall and shrinks the softmax by 2,427 entries.

3.2 Step B: v2-global decoder with anti-overfitting recipe

Same architecture as baseline, retrained on identical precomputed global features with the new recipe: AdamW weight_decay=0.01, embedding dropout 0.3 + padding_idx, EMA of weights (decay 0.999) used for validation/checkpointing, cosine LR schedule with 2-epoch warmup (min_lr 1e-5), label smoothing 0.1 kept, batch 512, patience 12.

Quantity	Baseline run	v2-global run
Best val loss (own scale)	4.7517 @ epoch 26	4.4205 @ epoch 27 (BPE scale, not comparable)
Best epoch vs stop	26 of 46 (gap widening)	27 of 39 (tight gap, gentle decline)
Val top-1 token accuracy	27.1%	30.9%
Train/val behaviour	val rising from epoch 27	still setting bests near epoch 27, slow rise after

Full-split evaluation (identical scoring code as baseline): greedy BLEU-1 0.5210 / BLEU-4 0.1187 (vs baseline greedy 0.4373 / 0.0959 - clean output without search); beam-5 alpha 1.2: BLEU-1 0.5358, BLEU-2 0.3383, BLEU-3 0.2092, BLEU-4 0.1373, ROUGE-L 0.2236. That already beat the tuned baseline (+9.6% BLEU-4) at half the beam width.

3.3 Step C: attention-v2 over spatial features (final winner)

Bahdanau attention decoder (embed 256, LSTM 512, attention dim 512; 11.37M params) consuming 49 locations x 1920 channels of pre-norm5 DenseNet201 spatial features, trained with the Step-B recipe plus scheduled sampling (teacher-forcing probability ramped from 1.0 to 0.75 across epochs 5-30) to fight exposure bias, ReduceLROnPlateau (factor 0.5, patience 3), batch 64, label smoothing 0.05. Best val loss 3.8161 at epoch 24, early-stopped at 36. Safe checkpointing throughout (experiments/v2_attention/, strict-improvement best.pt - the failure mode that destroyed the old attention run cannot recur).

Decode strategy	BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE-L
attention-v2 greedy (no search)	0.5523	0.3434	0.2143	0.1412	0.2472
attention-v2 beam-5, alpha 1.2	0.5649	0.3704	0.2447	0.1663	0.2548

For context, published Show-Attend-Tell-class models reach roughly BLEU-1 0.55-0.57 and BLEU-4 0.16-0.19 on Flickr8K; attention-v2 sits squarely in that territory using only this dataset. Its greedy decoding alone beats the baseline's best beam search.

4. Change-to-Metric Impact Map

Change	Why	Metrics affected
Beam sweep 5/0.6 -> 10/1.2	Search parameters were untuned	All BLEU up (+2.8% BLEU-4), no training needed
norm5 serving-bug fix	Inference must match training feature distribution	Prerequisite for valid attention serving; correctness fix
Checkpoint hygiene (strict-improvement saves)	Old runs silently overwrote good weights	Process safety - prevents recurrence of lost-model incident
Temperature calibration study	Verify what the confidence % means	ECE measured 1.3%; confirmed honest; prevented misguided 'fix'
BLIP PMI semantic scorer	Independent check for fluent-but-wrong captions	New capability: semantic_score; exposes decoder blind spots
BPE-6k subwords	Remove hapax memorisation targets	Top-1 accuracy 27.1% -> 30.9%; BLEU-3/4 up; smaller softmax
AdamW + emb dropout + EMA + cosine LR	Slow overfitting, stabilise weights	Deeper later val-loss minimum; overfit gap narrowed
Scheduled sampling 0->0.25	Close train/inference exposure mismatch	Free-running quality: greedy BLEU-4 0.0959 -> 0.1412
Spatial features + Bahdanau attention	Give decoder object locations, not just global gist	Largest single gain: BLEU-4 0.1373 -> 0.1663, ROUGE-L +14%

5. Production Deployment Changes

Item	Before	After
Active provider	notebook-tensorflow (Keras word-model)	attention (PyTorch attention-v2), auto-selected first; old chain retained as fallback
Checkpoint served	weights/model.h5 + tokenizer.pkl	experiments/v2_attention/best.pt + weights/bpe/tokenizer.json
Decode	TF beam-10 alpha 1.2 (word model)	torch beam-5 alpha 1.2 + repeat penalty (env-tunable)
API response	caption, confidence	+ semantic_score, semantic_pmi (BLIP PMI judge)
UI	Confidence bar only	+ Semantic match bar; tooltips explaining both signals

Live verification after deployment: public/test-image.jpg now captions as 'little boy is playing with yellow ball in the air' (semantic agreement 24.7%, previously 6.9% with 'little girl in red shirt is playing in the background'); backend/test_image2.jpg changed from the broken 'black dog is in the air in its mouth' to 'red airplane is in the air'. Health endpoint reports provider='attention', weights_loaded=true.

6. Engineering Findings (Evaluation Correctness)

Three bugs in newly written evaluation code were caught by refusing to accept implausible scores, each fixed against the production decoding semantics: (1) beam length normalisation must be $\text{GNMT}((5+\text{len})/6)^\alpha$, not len^α ; (2) unfinished prefixes must never be eligible as final hypotheses; (3) most importantly, the global decoder was trained on prefixes padded to MAX_LENGTH, so beam search must pad identically - feeding unpadded sequences silently shifted the input distribution and produced repetition-loop gibberish (BLEU 0.09 until fixed; 0.53 after, greedy unchanged throughout). Standing rule recorded in the repo: evaluation code must mirror production decoding byte-for-byte (padding, GNMT formula, repeat penalty).

7. Reproducibility - Where Everything Lives

Artefact	Location
BPE build (tokenizer, stats, encoded captions)	build_bpe.py -> weights/bpe/
v2-global training / results	train_v2.py -> experiments/v2_global/ (best.pt, eval_results.json, train_summary.json)
attention-v2 training / results	train_attention_v2.py -> experiments/v2_attention/ (best.pt, eval_results.json)
Beam sweep (16 configs)	beam_sweep.py -> experiments/beam_sweep/results.json
Calibration studies	fit_temperature.py, calibrate_semantic.py -> experiments/confidence/
Semantic scorer	inference/blip_scorer.py (+ test_pmi_validation.py)
Production provider	inference/attention_provider.py (auto-selected in inference/manager.py)
Baseline ground truth	weights/metadata.json (training history + evaluations)

Environment: Python 3.11 venv, PyTorch 2.5.1+cu121 (GTX 1080), TensorFlow 2.21 CPU (legacy fallback only), transformers 4.46.3, tokenizers 0.20.3, FastAPI + Uvicorn; frontend Next.js 16 / React 19 on port 3000, API on port 8000. Note: first request after server start costs about 13 s of DenseNet201 warm-up on CPU; subsequent requests are faster. Exact BLEU values may vary by roughly +/-0.003 across reseeds; the reported gains are far outside that noise band.